

Llano: evaluación de un estilo de escritura controlado para agentes de código en español

github.com/joaquinarroyo/llano

Corrida del 2026-10-05

Resumen

Llano es una skill para agentes de código que adapta ASD-STE100, el inglés técnico simplificado de la aviación, al español. Este piloto compara llano con una instrucción simple de concisión, con respuestas sin instrucción y con caveman. Usa 24 prompts técnicos en español y Claude Sonnet dentro de Claude Code.

Llano no acorta las respuestas frente a la instrucción de concisión: la diferencia en palabras es -1% (IC 95 %: -13% a $+8\%$). Su efecto está en la forma. Baja las violaciones de reglas de 1.57 a 0.32 cada 100 palabras, con menos violaciones en 20 de 24 prompts ($p < 0.0001$). También sube la legibilidad INFLESZ de 72.2 a 78.6. La fidelidad no cambia: llano conserva el 95 % de los hechos clave, igual que la instrucción de concisión. En la comparación a ciegas, el juez prefirió llano en 17 de 24 casos, pero la diferencia no es significativa ($p = 0.064$). El costo es la entrada: cada llamada suma unos 11126 tokens contra 2652.

1. Introducción

Los agentes de código producen cada vez más texto que una persona debe leer. Ese texto suele ser largo, repetitivo y lleno de fórmulas vacías. Andrej Karpathy propuso pedirle al modelo que escriba en ASD-STE100 (*Simplified Technical English*). Es un lenguaje controlado que la industria aeronáutica usa desde 1986 para los manuales de mantenimiento.

Llano adapta esa idea. Tiene un núcleo de reglas independiente del idioma y un paquete de idioma para el español neutro. Este informe mide si llano cumple tres objetivos:

1. Producir respuestas más cortas.
2. Producir respuestas que cumplen mejor las reglas de escritura clara.
3. No perder información en el proceso.

La comparación principal es contra una instrucción simple de concisión, no contra una respuesta sin instrucciones. El harness de evaluación de caveman mostró que comparar solo contra “sin instrucciones” infla el beneficio de un estilo, porque mezcla el estilo con el pedido genérico de brevedad.

2. Método

2.1. Grupos

Cada prompt se ejecuta en cuatro grupos. Todos usan el system prompt por defecto de Claude Code. Los grupos solo cambian el texto agregado con `--append-system-prompt`.

Grupo	Texto agregado
Sin instrucción	ninguno
Concisa	“Responde de forma concisa.”
Llano	“Responde de forma concisa.” + SKILL.md + paquete es
Caveman	“Responde de forma concisa.” + SKILL.md de caveman

2.2. Prompts y modelo

El conjunto tiene 24 prompts en español, en seis categorías: explicación, error, resumen, procedimiento, comparación y conceptual. Cada prompt incluye todo su contexto (logs, commits, hilos de discusión) y una lista de cinco hechos clave que una buena respuesta debe conservar.

El modelo generador es Claude Sonnet (**claude-sonnet-5-5**), con 1 corrida por combinación. Todas las llamadas usan Claude Code 2.1.289 en modo no interactivo (**claude -p**).

2.3. Aislamiento

Cada llamada corre sin la configuración del usuario: sin hooks, sin plugins, sin skills, sin servidores MCP y sin herramientas. Antes de cada corrida, un canario le pide al modelo que copie todas las instrucciones extra que recibió. La corrida se cancela si la respuesta menciona llano, caveman o context-mode. Un control positivo, sin aislamiento, confirmó que el canario detecta esas instrucciones cuando están presentes.

2.4. Métricas

Tokens visibles Tokens de salida informados por Claude, menos los tokens de razonamiento interno.

Palabras Palabras del texto en prosa. Se excluyen bloques de código, código en línea y tablas.

Violaciones cada 100 palabras Un linter determinístico cuenta frases largas (más de 25 palabras, o 20 en pasos numerados), punto y coma, párrafos de más de seis frases, muletillas prohibidas, palabras del diccionario a evitar, dudas apiladas y gerundios en cadena. El diccionario y las muletillas salen del paquete **es**.

INFLESZ Índice de legibilidad de Szigriszt-Pazos para el español. Un valor más alto indica un texto más fácil de leer. Entre 55 y 65 es “normal”.

Fidelidad Un juez (Claude Sonnet, también aislado) indica qué hechos clave conserva cada respuesta y cuenta las afirmaciones incorrectas.

Preferencia a ciegas El mismo juez compara llano contra cada otro grupo. Ve las dos respuestas como A y B, en orden aleatorio, y elige la que un desarrollador entiende mejor en una lectura sin perder información.

3. Resultados

3.1. Visión general

Grupo	Tokens visibles	Palabras	Viol./100 pal.	INFLESZ	Fidelidad	Errores
Sin instrucción	960	246	1.67	73.2	97 %	4
Concisa	622	158	1.57	72.2	95 %	3
Llano	590	156	0.32	78.6	95 %	3
Caveman	503	138	0.84	77.0	95 %	6

Tabla 1: Medianas por respuesta. Violaciones e INFLESZ son promedios. Fidelidad es el promedio de hechos clave conservados. Errores es el total de afirmaciones incorrectas.

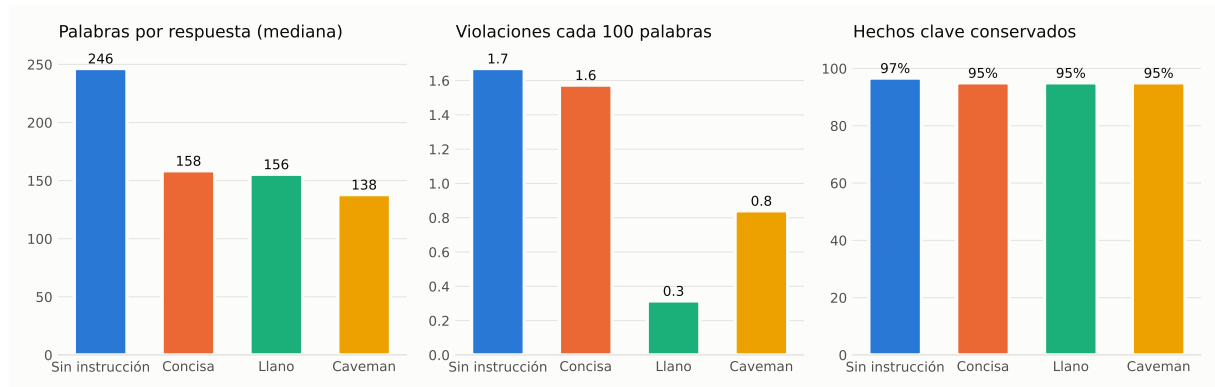


Figura 1: Largo, cumplimiento de reglas y fidelidad por grupo.

3.2. Diferencias pareadas

Para cada prompt se calcula el cociente entre llano y el grupo de referencia. La tabla muestra la mediana de esos cocientes y su intervalo de confianza del 95 % por bootstrap.

Llano contra	Palabras	Tokens visibles
Concisa	-1 % [-13 %, +8 %]	-2 % [-11 %, +2 %]
Sin instrucción	-39 % [-44 %, -29 %]	-38 % [-41 %, -30 %]
Caveman	+14 % [+3 %, +33 %]	+18 % [+7 %, +25 %]

3.3. Cumplimiento de reglas por tipo

Tipo	Sin instrucción	Concisa	Llano	Caveman
Frases largas	0.62	0.42	0.10	0.11
Punto y coma	0.32	0.47	0.00	0.09
Párrafos largos	0.00	0.00	0.00	0.00
Muletillas	0.06	0.05	0.00	0.00
Palabras a evitar	0.67	0.68	0.32	0.54
Dudas apiladas	0.00	0.00	0.00	0.00
Gerundios en cadena	0.00	0.00	0.00	0.00

Tabla 2: Violaciones cada 100 palabras, por tipo de regla.

3.4. Por categoría

Categoría	Palabras (mediana)		Viol./100 pal.	
	Concisa	Llano	Concisa	Llano
Comparación	176	154	1.21	0.18
Conceptual	251	200	2.21	0.29
Error	155	160	1.38	0.70
Explicación	268	240	0.84	0.58
Procedimiento	150	152	1.79	0.14
Resumen	94	64	2.01	0.00

Tabla 3: Llano contra la instrucción de concisión, por categoría de prompt.

3.5. Preferencia a ciegas

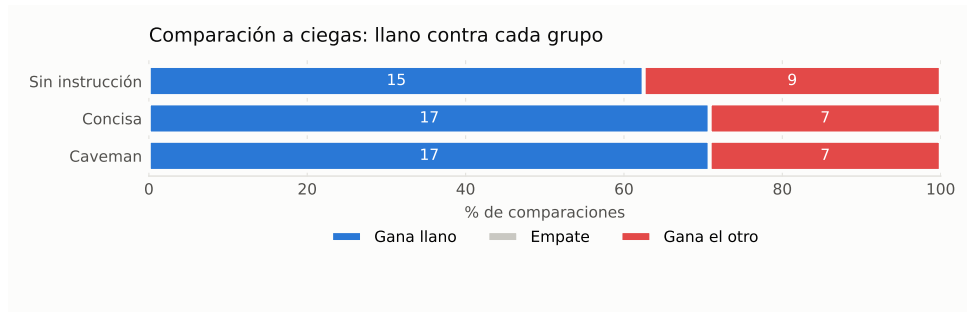


Figura 2: Resultado de la comparación a ciegas.

Llano contra	Gana llano	Empate	Gana el otro	n	IC 95 % victorias	p
Sin instrucción	15	0	9	24	43–79 %	0.307
Concisa	17	0	7	24	51–85 %	0.064
Caveman	17	0	7	24	51–85 %	0.064

Tabla 4: Comparaciones a ciegas, en número de prompts.

3.6. Costo de entrada

Llano agrega su SKILL.md y el paquete de idioma a cada llamada. La entrada mediana por llamada es 11126 tokens con llano y 2652 con la instrucción de concisión. En una sesión real la mayor parte de ese costo se cachea, pero ocupa contexto en cada turno.

4. Discusión

Llano no es una herramienta para ahorrar tokens. Frente a la instrucción de concisión, el largo no cambia: -1% en palabras y -2% en tokens visibles, con intervalos que incluyen el cero. Toda la reducción frente a la respuesta sin instrucción (-39% en palabras) la consigue ya la frase “Responde de forma concisa”. Caveman produce respuestas más cortas que llano: llano usa $+14\%$ más palabras. Este resultado coincide con la lección del harness de caveman: casi todo el ahorro de un estilo viene del pedido genérico de brevedad.

El efecto de llano está en la forma. Las violaciones cada 100 palabras bajan de 1.57 a 0.32. Llano tiene menos violaciones que la instrucción de concisión en 20 prompts, las mismas en 3 y más en 1 ($p < 0.0001$ en una prueba de signos). Las frases largas, el punto y coma y las muletillas casi desaparecen. Contra caveman la diferencia también es clara: 15 prompts mejor, 2 peor ($p = 0.0023$). La legibilidad INFLESZ sube de 72.2 a 78.6, de “bastante fácil” hacia el borde de “muy fácil”.

Esta métrica tiene un sesgo a favor de llano: el linter mide las reglas que llano conoce. Por eso importan las dos métricas siguientes.

Llano no pierde información. La fidelidad es 95 % con llano, 95 % con la instrucción de concisión y 97 % sin instrucción. Los hechos que faltan son casi siempre detalles secundarios, como la versión desplegada o el paso exacto del Dockerfile. Las afirmaciones incorrectas son pocas en todos los grupos. Caveman tiene el doble que llano.

La preferencia a ciegas favorece a llano, pero sin significancia. El juez prefirió llano en 17 de 24 comparaciones contra la instrucción de concisión y en 17 contra caveman ($p = 0.064$ en ambos casos). Contra la respuesta sin instrucción ganó 15 veces ($p = 0.307$). Con 24 prompts y una sola corrida, el piloto no alcanza para confirmar esta preferencia.

El costo es la entrada. Llano agrega su skill y el paquete de idioma a cada llamada: unos 11126 tokens contra 2652. Con la caché de prompts, ese costo baja mucho en una sesión real, pero el texto sigue ocupando contexto. Reducir el paquete, por ejemplo cargando el diccionario solo en modo `all`, es la mejora más directa.

Próximos pasos. Ejecutar tres corridas con Opus, Sonnet y Haiku para medir la variación y evitar que el juez evalúe solo a su propio modelo. Ampliar el conjunto de prompts con casos escritos por otras personas. Medir una versión de llano con un paquete más corto.

5. Limitaciones

- **Una corrida, un modelo.** Este es un piloto. La variación entre corridas no está medida.
- **Juez de la misma familia.** El juez y el generador son modelos Claude. Un juez puede preferir el estilo de su propia familia.
- **Linter heurístico.** El linter cuenta lo que una expresión regular puede ver. No detecta el “se” impersonal ni el uso de “el mismo” como pronombre. El conteo de sílabas para INFLESZ es aproximado.
- **El linter usa las reglas de llano.** El grupo llano conoce esas reglas y los demás grupos no. Por eso la métrica de violaciones mide adherencia a llano, no calidad general. La fidelidad y la preferencia a ciegas compensan ese sesgo.
- **Prompts propios.** Los prompts los escribió el autor de la skill.

6. Reproducción

```
cd evals
uv run run.py --run-id <id> --models sonnet --runs 1
uv run judge.py --run-id <id> --judge-model sonnet
uv run measure.py --run-id <id>
cd report && tectonic report.tex
```

Las respuestas, los veredictos del juez y el resumen quedan en `snapshots/`, `judge/` y `results/`.